

Analysis of Randomized Quicksort

Average-Case Time Complexity of Randomized Quicksort

Randomized Quicksort is an upgrade over the standard Quicksort algorithm because it picks the pivot element uniformly at random from the current subarray during every single recursive call. Unlike deterministic Quicksort where a bad pivot choice can repeatedly give us highly unbalanced partitions Randomized Quicksort heavily cuts down the chances of consistently picking terrible pivots. Because of this, the algorithm easily achieves an expected average-case time complexity of $O(n \log n)$, no matter how the initial input array is ordered.

The total running time of Randomized Quicksort mostly depends on two main operations that happen recursively:

- **Partitioning the array:** This part takes linear time, which is just $O(n)$.
- **Recursively sorting the subarrays:** Sorting the left and right subarrays that we get after the partitioning step.

So, the overall running time completely depends on how evenly our randomly selected pivot manages to split the array into two smaller subproblems.

Recurrence Relation Analysis

Let $T(n)$ represent the expected running time required to sort an array containing n elements.

Each recursive call performs a partition operation requiring: **$O(n)$ time.**

Since every element has an equal probability of being selected as the pivot, every possible partition is equally likely. Therefore, the expected running time can be expressed as:

$$E[T(n)] = \frac{1}{n} \sum_{k=0}^{n-1} (E[T(k)] + E[T(n-k-1)]) + cn$$

where:

- k represents the number of elements placed in the left partition,
- $n-k-1$ represents the number of elements placed in the right partition,
- cn represents the linear cost of partitioning.

Since the two recursive subproblems are symmetric, the recurrence can be simplified as:

$$E[T(n)] = \frac{2}{n} \sum_{k=0}^{n-1} E[T(k)] + cn$$

Solving this recurrence using substitution or mathematical induction gives:

$$E[T(n)] = O(n \log n)$$

Therefore, the expected average-case running time of Randomized Quicksort is:

$$O(n \log n)$$

Indicator Random Variable Analysis

Another common method for analyzing Randomized Quicksort is through **indicator random variables**, which estimate the expected number of comparisons performed during execution.

Consider two elements x_i and x_j in the sorted order.

Define the indicator random variable:

$$X_{ij} = \begin{cases} 1, & \text{if } x_i \text{ and } x_j \text{ are compared} \\ 0, & \text{otherwise} \end{cases}$$

The total number of comparisons performed by the algorithm is:

$$X = \sum_{i < j} X_{ij}$$

Using the linearity of expectation,

$$E[X] = \sum_{i < j} E[X_{ij}]$$

Two elements are compared only if one of them is chosen as the first pivot among all elements lying between them. The probability of this occurring is

$$P(X_{ij} = 1) = \frac{2}{j - i + 1}$$

Substituting this probability into the expected value calculation gives

$$E[X] = O(n \log n)$$

Since each comparison requires constant time, the expected running time of Randomized Quicksort is also $O(n \log n)$

Why Randomized Quicksort Achieves $O(n \log n)$

Randomized Quicksort easily hits an expected average-case time complexity of $O(n \log n)$ because picking the pivot randomly heavily cuts down the chances of repeatedly getting highly unbalanced partitions. Even if a single partition turns out to be uneven now and then, the probability of consistently choosing terrible pivots throughout the entire recursion is next to nothing.

Consequently, most of our recursive calls end up producing reasonably balanced partitions, which keeps the total recursion depth at around $\log n$. Since each level of recursion processes all n elements during the partitioning phase, and we only have roughly $\log n$ levels, the total expected running time simply becomes:

$$O(n) * O(\log n) = O(n \log n)$$

This makes Randomized Quicksort way more robust than Deterministic Quicksort, especially when we have absolutely no idea what the input distribution looks like beforehand.

Conclusion

The theoretical analysis clearly shows that Randomized Quicksort easily hits an expected average-case time complexity of $O(n \log n)$, simply because it picks the pivot uniformly at random during every single recursive call. Both the recurrence relation and the indicator random variable analyses prove that the expected number of recursive operations and element comparisons only grows proportionally to $n \log n$.

Now, of course, the algorithm can still technically hit a worst-case running time of $O(n^2)$ if we get extremely unbalanced partitions, but thanks to the random pivot selection, the chances of this happening in real life are practically next to nothing. Consequently, Randomized Quicksort gives us way more consistent and reliable performance compared to Deterministic Quicksort across all kinds of different input distributions.

Analysis of Hashing with Chaining

Expected Time Complexity Under Simple Uniform Hashing

Under the assumption of simple uniform hashing, every key is equally likely to be placed into any bucket in the hash table, regardless of where other keys are stored. This assumption helps distribute the keys evenly across the available buckets, reducing the likelihood of long chains caused by collisions.

Let:

- n = number of elements stored in the hash table
- m = number of buckets (slots)

The load factor is defined as:

$$\alpha = n / m$$

where α represents the average number of elements stored in each bucket.

Insert Operation

For the insert operation, the code first calculates the hash value to find the correct bucket and then pushes the new key-value pair into that corresponding chain. If the key is already present, it simply updates its associated value.

Under simple uniform hashing, the expected number of elements we need to check inside a bucket is directly proportional to the load factor α . Therefore, the expected time complexity for insertion comes out to be:

$$O(1+\alpha)$$

When we maintain this load factor as a small constant through dynamic resizing, the expected insertion time simply becomes:

$$O(1)$$

Search Operation

For searching, we begin by calculating the hash value of the given key to locate the correct bucket. The algorithm then simply traverses through the chain until it finds the required key or hits the end of the chain. Since the expected length of the chain is exactly equal to the load factor α , the expected search time comes out to be:

$$O(1+\alpha)$$

If we keep the load factor small, the expected search time effectively becomes:

$$O(1)$$

Delete Operation

Deletion first searches for the specified key and then removes that key-value pair from the chain. Because the searching part takes up most of the effort, the expected deletion time is also:

$$O(1+\alpha)$$

When the load factor is kept low, deletion also easily executes in expected constant time.

Effect of the Load Factor

The load factor is honestly one of the most critical factors affecting our hash table's performance. It represents the average number of elements stored in each bucket. A small load factor means fewer elements are sitting in each bucket, which gives us shorter chains and way faster insertion, search, and deletion operations. But as the load factor increases, more collisions start happening, which makes the chains longer and heavily increases the number of comparisons needed to find a key.

For example:

Load Factor(α)	Expected Performance
0.25	Excellent (very short chains)
0.50	Very Good
0.75	Good
Greater than 1.0	Performance begins to degrade due to longer chains

Maintaining the load factor below approximately **0.75** provides a good balance between memory usage and operational efficiency.

Strategies for Maintaining a Low Load Factor and Minimizing Collisions

Several techniques can be used to maintain efficient hash table performance and reduce collisions:

1. Dynamic Resizing

This is honestly one of the most effective strategies out there. When the load factor crosses a specific threshold (usually 0.75), we increase the size of the hash table typically by doubling the number of buckets. After that, all the existing elements are rehashed and inserted into the new table. This process instantly cuts down the average chain length and brings the performance back to top speed for future operations.

2. Choosing an Effective Hash Function

A good hash function should distribute the keys as uniformly as possible across all the available buckets. In this implementation, I've used Python's built-in `hash()` function along with the modulo operator to map keys to valid bucket indices. A well-designed hash function heavily minimizes collisions by making sure keys don't just pile up inside a few specific buckets.

3. Chaining for Collision Resolution

Chaining allows us to store multiple key-value pairs inside the exact same bucket by simply maintaining a separate list for each slot. So even when collisions happen, no data is lost, and all elements remain fully accessible. The expected performance stays super-efficient as long as we keep the load factor reasonably low.

Conclusion

Under the assumption of simple uniform hashing, the insert, search, and delete operations all have an expected time complexity of $O(1)$, provided the load factor is maintained as a small constant. As the load factor shoots up, collisions become way more frequent, which leads to longer chains and kills the performance.

By combining dynamic resizing with an effective hash function and using chaining for collision resolution, we can easily maintain a low load factor, minimize collisions, and ensure that our hash table keeps running smoothly even as the number of stored elements grows.